



RUTA 5: C# → Unity (160 horas)

Herramientas: Visual Studio Community, Unity Personal

Objetivo de la ruta: Formar desarrolladores capaces de diseñar e implementar videojuegos 2D estructurados, aplicando programación orientada a objetos en C# e integrando correctamente scripts, componentes, interfaz y arquitectura interna de Unity bajo un flujo de trabajo profesional.

MÓDULO: Fundamentos de C# Aplicados a Videojuegos

Duración: 40 horas

Competencia: Aplica estructuras fundamentales de programación en C# para modelar mecánicas básicas de videojuego.

Tema 1. Tipos de datos y estructuras básicas

- 1.1 Tipos primitivos
 - 1.1.1 Declaración de variables
 - 1.1.2 Inicialización
 - 1.1.3 Alcance de variables
 - 1.1.4 Conversión implícita y explícita
- 1.2 Diferencias entre float y double en físicas
 - 1.2.1 Precisión decimal
 - 1.2.2 Uso en cálculos de movimiento
 - 1.2.3 Impacto en rendimiento
- 1.3 Constantes y casting
 - 1.3.1 Uso de const
 - 1.3.2 Casting explícito
 - 1.3.3 Parseo de datos
- 1.4 Arreglos
 - 1.4.1 Declaración y tamaño fijo
 - 1.4.2 Recorrido con for
 - 1.4.3 Manipulación de posiciones
- 1.5 List<T>
 - 1.5.1 Inicialización dinámica
 - 1.5.2 Métodos Add y Remove
 - 1.5.3 Búsqueda y validación
- 1.6 Diccionarios para inventarios simples
 - 1.6.1 Clave-valor
 - 1.6.2 ContainsKey
 - 1.6.3 Actualización de cantidades



Tema 2. Métodos y control de flujo

- 2.1 Declaración y retorno
 - 2.1.1 Métodos void
 - 2.1.2 Métodos con retorno
 - 2.1.3 Llamado entre métodos
- 2.2 Parámetros por valor y referencia
 - 2.2.1 Paso por copia
 - 2.2.2 Uso de ref
 - 2.2.3 Uso de out
- 2.3 Sobrecarga
 - 2.3.1 Métodos con distintos parámetros
 - 2.3.2 Diseño limpio
- 2.4 Condicionales
 - 2.4.1 if – else
 - 2.4.2 Operadores lógicos
 - 2.4.3 Validaciones compuestas
- 2.5 Switch
 - 2.5.1 Estructura básica
 - 2.5.2 Uso con enum
- 2.6 Ciclos
 - 2.6.1 for
 - 2.6.2 while
 - 2.6.3 foreach
 - 2.6.4 break y continue
- 2.7 Manejo básico de excepciones
 - 2.7.1 try–catch
 - 2.7.2 Excepciones comunes
 - 2.7.3 Validación preventiva

Tema 3. Introducción a POO

- 3.1 Clase y objeto
 - 3.1.1 Estructura de clase
 - 3.1.2 Instanciación
 - 3.1.3 Multiplicidad de objetos
- 3.2 Propiedades
 - 3.2.1 Get y Set
 - 3.2.2 Modificadores de acceso
- 3.3 Constructores
 - 3.3.1 Constructor por defecto
 - 3.3.2 Constructor parametrizado
- 3.4 Encapsulamiento
 - 3.4.1 Campos privados
 - 3.4.2 Métodos públicos controlados



Actividad Integradora: Desarrollar en consola un sistema estructurado de personaje RPG que incluya:

- Clase Personaje con vida, ataque y defensa.
- Inventario usando List o Dictionary.
- Método RecibirDaño().
- Validación de muerte.
- Reporte dinámico en consola.
- Separación en múltiples clases.

MÓDULO 2. Programación Orientada a Objetos y Arquitectura de Juego

Duración: 30 horas

Competencia: Diseña sistemas estructurados aplicando herencia, polimorfismo y control de estados.

Tema 1. Herencia y polimorfismo

- 1.1 Clase base Entidad
 - 1.1.1 Atributos comunes
 - 1.1.2 Métodos compartidos
- 1.2 Clases derivadas
 - 1.2.1 Jugador
 - 1.2.2 Enemigo
 - 1.2.3 Especialización de comportamiento
- 1.3 Override
 - 1.3.1 Virtual
 - 1.3.2 Sobrescritura segura
- 1.4 Polimorfismo aplicado a combate
 - 1.4.1 Referencias tipo base
 - 1.4.2 Llamado dinámico
 - 1.4.3 Sistema de daño escalable

Tema 2. Clases abstractas e interfaces

- 2.1 Métodos abstractos
 - 2.1.1 Declaración
 - 2.1.2 Implementación obligatoria
- 2.2 Interfaces
 - 2.2.1 Declaración
 - 2.2.2 Implementación múltiple
 - 2.2.3 Interfaces para interacción

Tema 3. Simulación avanzada

- 3.1 Enum para estados
 - 3.1.1 Declaración
 - 3.1.2 Integración en lógica



- 3.2 Máquina de estados
 - 3.2.1 Cambio de estados
 - 3.2.2 Validación de transiciones
- 3.3 Sistema por turnos
 - 3.3.1 Orden de acciones
 - 3.3.2 Control de flujo
- 3.4 Validación de victoria/derrota
 - 3.4.1 Condiciones finales
 - 3.4.2 Reinicio estructurado

Actividad Integradora: Desarrollar una simulación completa de combate estructurado que incluya:

- Clase abstracta Entidad.
- Jugador y múltiples enemigos.
- Sistema de estados con enum.
- Polimorfismo en el método Atacar().
- Validación automática de victoria o derrota.
- Código modular y escalable.

MÓDULO 3. Desarrollo 2D y Dominio de la Interfaz en Unity

Duración: 40 horas

Competencia: Comprende la arquitectura de Unity y desarrolla mecánicas 2D básicas integrando scripts y componentes.

Tema 1. Instalación y configuración profesional

- 1.1 Unity Hub
 - 1.1.1 Gestión de versiones
 - 1.1.2 Instalación de módulos
- 1.2 Instalación LTS
 - 1.2.1 Estabilidad
 - 1.2.2 Compatibilidad
- 1.3 Creación de proyecto 2D
 - 1.3.1 Configuración inicial
 - 1.3.2 Ajustes básicos
- 1.4 Integración con Visual Studio
 - 1.4.1 Configuración externa
 - 1.4.2 Debug básico

Tema 2. Interfaz completa

- 2.1 Hierarchy
 - 2.1.1 Estructura padre-hijo
 - 2.1.2 Organización lógica



2.2 Scene

2.2.1 Manipulación 2D

2.2.2 Herramientas

2.3 Game

2.3.1 Resolución

2.3.2 Ejecución

2.4 Inspector

2.4.1 Componentes

2.4.2 Serialización

2.5 Project

2.5.1 Carpetas profesionales

2.5.2 Organización modular

2.6 Console

2.6.1 Debug.Log

2.6.2 Identificación de errores

Tema 3. Arquitectura y programación básica

3.1 GameObjects y componentes

3.1.1 Sistema basado en componentes

3.1.2 Agregar scripts

3.2 Transform

3.2.1 Posición

3.2.2 Rotación

3.2.3 Escala

3.3 Prefabs

3.3.1 Creación

3.3.2 Instanciación

3.4 Ciclo de vida

3.4.1 Awake

3.4.2 Start

3.4.3 Update

3.5 Movimiento lateral

3.5.1 Input.GetAxis

3.5.2 Time.deltaTime

Actividad Integradora: Construir un prototipo 2D funcional que incluya:

- Personaje con movimiento lateral fluido.
- Uso correcto de prefab.
- Organización profesional del proyecto.
- Script estructurado y conectado.
- Depuración en Console.



MÓDULO 4. Videojuego 2D Completo y Flujo Profesional

Duración: 40 horas

Competencia: Integra físicas, interfaz y gestión de escenas para desarrollar un videojuego 2D completo.

Tema 1. Física aplicada al videojuego

1.1 Rigidbody2D

1.1.1 Masa

1.1.2 Gravedad

1.1.3 Fuerzas

1.2 Sistema de colisiones

1.2.1 BoxCollider2D

1.2.2 CircleCollider2D

1.2.3 PolygonCollider2D

1.3 Triggers

1.3.1 IsTrigger

1.3.2 Eventos de activación

1.4 Detección de impactos

1.4.1 OnCollisionEnter2D

1.4.2 Validación por tag

1.4.3 Respuesta a impacto

1.5 Movimiento con físicas

1.5.1 Salto con fuerza vertical

1.5.2 Detección de suelo

1.5.3 Ajuste de gravedad para jugabilidad

Tema 2. Interfaz de usuario

2.1 Canvas

2.1.1 Render Mode

2.1.2 Escalado

2.2 Text / Image

2.2.1 Actualización dinámica

2.2.2 Referencias desde script

2.3 Button

2.3.1 Eventos OnClick

2.3.2 Cambio de escenas

Tema 3. Gestión de flujo del juego

3.1 SceneManager

3.1.1 Carga simple

3.1.2 Reinicio

3.2 Menú principal

3.2.1 Navegación



3.3 Pantalla final

3.3.1 Victoria

3.3.2 Derrota

Tema 4. Organización avanzada

4.1 Comunicación entre scripts

4.1.1 GetComponent

4.1.2 Referencias públicas

4.2 Separación de responsabilidades

4.2.1 Script jugador

4.2.2 Script enemigo

4.2.3 Script GameManager

4.3 Buenas prácticas

4.3.1 Nombres claros

4.3.2 Código modular

Actividad Integradora: Desarrollar un nivel completo tipo plataforma que incluya

- Movimiento y salto con físicas reales.
- Enemigos funcionales.
- Sistema de vida y puntaje.
- HUD dinámico.
- Menú principal y pantalla final.
- Arquitectura limpia y modular.

PROYECTO FINAL RUTA GODOT – DESARROLLO DE VIDEOJUEGO 2D

Desarrollar un videojuego 2D completo que incluya:

- Menú principal funcional
- Nivel jugable
- Personaje controlable
- Sistema de movimiento vectorial
- Sistema de animaciones por estados (idle, movimiento y daño)
- Sistema de vida
- Enemigos con comportamiento (patrullaje, detección y ataque)
- Sistema de puntaje visible en pantalla
- Condición de victoria
- Pantalla de victoria
- Pantalla de derrota
- Cambio de escenas funcional
- Persistencia básica de datos



Puntos de Vinculación con Talleres Creativos

Estas vinculaciones permiten complementar el desarrollo técnico del videojuego con elementos narrativos, visuales y sonoros desarrollados en otros talleres de la Escuela de Videojuegos.

Vinculación con Literatura para Videojuegos

Momento sugerido: Después del **Módulo 2**

Propósito:

- Definir la idea narrativa del videojuego
- Establecer el objetivo del jugador
- Construir el contexto del mundo del juego

Vinculación con Dibujo para Videojuegos

Momento sugerido: Durante el **Módulo 3**

Propósito:

- Diseño del personaje principal
- Diseño de enemigos o elementos interactivos
- Diseño del escenario del videojuego
- Definición del estilo visual del juego

Vinculación con Música para Videojuegos

Momento sugerido: Durante el **Módulo 4 o antes del Proyecto Final**

Propósito:

- Integración de música ambiental
- Incorporación de efectos de sonido
- Retroalimentación sonora durante la interacción del jugador